Alexander Wichern
3231583
EECS 581

The source code for the program was generated with Sonnet 5 (medium) on 9/27/26. The prompt used was:

Problem description

Write a C or C++ program that reads a line of text and extracts a single valid IPv4 address — optionally followed by a port number — embedded anywhere in that text. Only digits, periods (.), and colons (:) are ever part of a valid token; every other character is garbage and is skipped. A candidate token must match the address grammar in full — no partial matches, no truncating to find a valid piece inside a longer run.

An address is four octets separated by periods (octet.octet.octet.octet), each octet 1–3 digits, value 0–255, no leading zero unless the value is exactly 0. An optional :port may follow the fourth octet: 1–5 digits, value 0–65535, same leading-zero rule. If a colon is present, the port must be fully valid or the entire match — address included — is rejected.

Function prototype

The function returns whether a valid address was found, and delivers the address and port through two additional parameters.

C:

```
// Returns 1 if a valid address was found, 0 otherwise.
// On success: *outAddress holds the 32-bit value, and
// *outPort holds the port number, or -1 if no port was present.
// On failure: *outAddress is set to 0 and *outPort is set to -1.
int extractIPv4(const char* str, unsigned long* outAddress, int* outPort);
```

C++

```
// Returns true if a valid address was found, false otherwise.
// On success: outAddress holds the 32-bit value,
// and outPort holds the port number, or -1 if no port was present.
// On failure: outAddress is set to 0 and outPort is set to -1.
bool extractIPv4(const std::string& str, unsigned long& outAddress, int& outPort);
```

Built-in functions/libraries you may not use

- Any string-to-number conversion function: atoi, atol, atoll, strtol, strtoul, strtod, stoi, stol, stoul, sscanf, scanf with numeric conversions.
- Any address-parsing library function: inet_aton, inet_pton, inet_addr, or equivalents.
- Any regular-expression facility (std::regex, POSIX regex.h, or similar) — the parsing and validation logic must be your own character-by-character code, not a pattern matched by a library.
- Standard character-classification functions (isdigit, etc.) are fine to use.

Other requirements

- Exactly one valid address may be extracted per input line; everything else in the line is either garbage (skipped) or part of a candidate token that fails validation.

- Reject anything that does not exactly match the grammar above — wrong octet count, empty octet, out-of-range octet or port, a disallowed leading zero, a second colon, a colon not immediately after the fourth octet, or a stray period/colon directly adjacent to an otherwise-valid address.
- On success, print: Extracted IPv4 address: A.B.C.D (decimal value: N, port: P) where N is the 32-bit decimal value and P is the port number or the literal text none.

Program requirements

- Input loop: main continuously prompts the user for input until the user enters END (case-sensitive), then prints Program terminated. and exits.
- Extraction function: implement extractIPv4 exactly as prototyped above. All digit accumulation must be done by hand.
- Display: main receives the result from extractIPv4 and formats the output exactly as specified above.

You are an expert C (write the program in C) software engineer with 10 years of experience, build the program described above

After my own examination and brief testing of the code I noticed it failed to handle cases in which a valid ipv4 address had either a leading or tailing “.” or “:”. After noticing that I ran the second prompt: “this program has an issue where leading or trailing . cause hte ipv4 address to not be detected, diagnos this problem fix it and write and run a robust set of test cases to validate, show me the test cases”. This prompt failed to produce test cases for some reason but made functioning changes to the code for “.” I then added the criteria for “:” detection myself. From there at least with the testing I conducted the program is able to parse ipv4 out of any string.

Before main passes the users input to extractIPv4() and has a loop that trims of any lingering newline characters, main then passes the trimmed input to extractIPv4 along with the address of addr and port which are initialized earlier in main

extractIPv4() then checks that it was not passed a null string before beginning to evaluate tokens in the users string, skipping any elements that are not a digit, a “.”, and a “:” When the iterator finds a digit it starts building a potentially valid token noting when the token started and its length with explicit checks to trim the length if a trailing “.” or “:” are found. The function then makes a call to parseToken() passing in the starting index of the token, its length, the address of mains address variable, the addres of mains port variable, and an array to store the binary octets from the parsed token.

parseToken() then iterates through for octet, making a call to parseNumber for each(). parseNumber() then sums the 3 numbers it was passed in to validate that they are in range. The function does this by multiplying the current value by ten and adding the next digit to it. For 192 1, 1 * 10 +9, 19, 19*10 +2, 192, getting the value 192 which it then checks is less than maxValue and returns true. Back in parseToken() the function then validates that the element at the next position is indeed a “.” or else it returns false. After checking 3 octets it skips the check for the trialing “.” and begins to look for a port, the function does this by making another all to parseNumbers, this time instead of looking for a valid

octet value (len 3, 0-255) it looks for a valid port value (len 5, 65535). However if the current position in the string is equal to the current length of the string it detects that there is nothing left to read and doesn't bother to check for a port. If all octets/port are found to be valid extractIPv4() the function returns the decimal value of the ipv4 address the integer value of the port and the array of octets back to main. Main the prints the results out to the user and looks for another input.